

OPERATIONS

HORIZON GRID

Operations Guide

Version: 0.1.0

Documentation prepared: 2026-08-17

PREPARED FOR EVALUATION

Table of Contents

Operations Guide	3
------------------	---

Operations Guide

This chapter is written for the person who has to keep a running HORIZON GRID install healthy day to day -- not the person building it. It answers "what is actually running on this machine," "how do I check it's okay without reading code," "how does the cache/background job actually behave," "how do I back this up," and "what do I actually watch on the dashboards." The full technical detail behind each of these -- Celery internals, Redis key schemes, the Compose/Kubernetes deployment topology, the complete environment-variable reference -- already exists in the **Background Processing and Caching** and **Deployment and Troubleshooting** chapters; this chapter summarizes only what an operator needs and points back to those chapters rather than re-deriving them.

1. What's actually running

A Windows-installed platform is, underneath the Start Menu shortcuts, eight Docker containers defined by `docker-compose.yml` (plus `docker-compose.prod.yml` as a production override -- see the Deployment chapter for exactly what that override changes). None of them carry an explicit `container_name:` in the compose files, so Docker Compose names them by its own default rule, `<project>-<service>-<replica>`. Because the installer copies the platform into a directory literally named `app` (`%ProgramFiles%\IOC Intelligence Platform\app\`), the Compose project name is `app`, and the real container names on an installed copy are:

Container	Image / build	Role	Restart policy
<code>app-backend-1</code>	built from <code>backend/</code>	FastAPI API, port 8000 -- runs <code>alembic upgrade head</code> then Uvicorn on every start	none -- a crash stays down until recreated
<code>app-frontend-1</code>	built from <code>frontend/</code>	Next.js UI, port 3000	none
<code>app-postgres-1</code>	<code>postgres:16-alpine</code>	primary datastore -- investigations, cases, users, runtime provider config	none
<code>app-redis-1</code>	<code>redis:7-alpine</code>	provider-result cache, lookup rate limiter, Celery broker + result backend (three logical DB indexes on one process)	none
<code>app-neo4j-1</code>	<code>neo4j:5-community</code>	correlation graph (malware/threat-actor/campaign/CVE/MITRE-technique relationships)	none
<code>app-opensearch-1</code>	<code>opensearchproject/opensearch:2.17.0</code>	search/indexing backend	none
<code>app-celery_worker-1</code>	same backend image, <code>command: celery ... worker</code>	executes the one scheduled task (see §3)	<code>unless-stopped</code>
<code>app-celery_beat-1</code>	same backend image, <code>command: celery ... beat</code>	fires that task's schedule every hour; runs no task itself	<code>unless-stopped</code>

The asymmetry in the last column is deliberate, not an oversight: only the two Celery containers auto-restart. `backend`, the four datastores, and `frontend` do not, so a crashed backend or database stays down until an operator (or the wizard's own polling during install) explicitly brings it back -- `docker compose up -d <service>`, or the **Start/Restart Platform** Start Menu shortcuts, which wrap that same command.

All four datastores publish their host ports to `127.0.0.1` only (Postgres 5433, Redis 6379, Neo4j HTTP 7475/Bolt 7688, OpenSearch 9200, on this install -- see the real-ports note in the System Overview chapter). Only `backend` (8000) and `frontend` (3000) are published on every interface, since those are the two services meant to be reachable from other machines on the LAN.

[MISSING FIGURE: standalone-operations-guide-diagram-1.png]

2. Day-to-day health checks

Four independent ways to answer "is the platform okay right now," from fastest/shallowest to slowest/deepest:

1. **GET /health** (e.g. `http://localhost:8000/health`, no auth, outside `/api/v1`) -- the shallowest possible check. It returns `{"status":"ok","service":"HORIZON GRID"}` the instant Uvicorn accepts connections, and it does **not** check Postgres, Redis, Neo4j, or OpenSearch -- only that the backend process itself is up. This is exactly what the Setup Wizard polls for up to three minutes after `docker compose up`, and what a Kubernetes readiness/liveness probe would hit in that topology. A failure here means the backend process itself is down or unreachable; it says nothing about whether the datastores behind it are healthy.
2. **Service Status** (Start Menu shortcut, wraps `Service-Status.ps1`) -- the one-click version of the above, built for an administrator who has never typed a Docker command. It runs `docker compose ps` against both compose files, then separately checks the backend health endpoint and the frontend, and reports Docker Desktop's own running state -- three independent yes/no answers in one screen (containers listed, `Backend API: OK/NOT RESPONDING`, `Web interface: OK/NOT RESPONDING`, `Docker Desktop: Running/NOT RUNNING`).
3. **Diagnostics** (Start Menu shortcut, wraps `Diagnostics.ps1`) -- the deeper option when Service Status shows a problem. It zips `docker compose ps`, the last 200 lines of every container's logs, the setup log, a prerequisite-check run, and a redacted copy of `.env` (variable names and character counts only, e.g. `VIRUSTOTAL_API_KEY=***REDACTED*** (set, 32 chars)` -- the real `.env` is never included) into a single dated file on the Desktop. It's written to be handed to someone else for support without exposing a single real credential.
4. **GET /api/v1/providers/health** and **GET /api/v1/dashboard/kpis** -- the application-level view, covered in §6 below, for "is the platform's actual intelligence-gathering work succeeding," as distinct from "is the container up."

For anyone comfortable at a terminal, the same information Service Status gathers is `docker compose -f docker-compose.yml -f docker-compose.prod.yml ps` and `docker compose ... logs <service>` run from the install directory -- structured JSON logs (`structlog`, configured before anything else in `app/main.py`), so a startup failure lands in the same greppable format as a runtime one.



```

Pretty-print
{"status":"ok","service":"IOC Intelligence Platform"}

```

Figure 2 — The Service Status shortcut's console output: container list, Backend API / Web interface health checks, and Docker Desktop state, each an independent yes/no.

3. How provider-result caching works, operationally

Every IOC provider's result is cached in Redis (logical DB `/0`, shared with the lookup rate limiter) for **one hour by default** (`provider_cache_ttl_seconds`, `config.py:126`). The two operational facts worth internalizing:

- **The cache is positive-only.** A write only happens when a provider's call actually succeeds (`ProviderStatus.OK`). A provider that currently has no data for an IOC (`NO_DATA`), or that errored, times out, or was rate-limited, is never cached -- it gets queried again, in full, on every subsequent lookup of that same IOC until it eventually returns `OK`. There is no negative-result caching and no cache metrics/hit-rate counter exposed anywhere -- the only per-lookup signal is the `from_cache` boolean on each provider result.

- **There is no manual invalidation.** No "force refresh" flag exists on the investigation endpoint and no cache-busting API exists. If a provider's underlying data has genuinely changed and a fresher read is needed sooner than the hour, the only lever available today is waiting out the TTL.

The practical, user-visible symptom of this design -- already documented from the analyst's side in the Health & Troubleshooting chapter -- is that a repeat lookup on the same IOC within the hour comes back noticeably faster. That's this cache working as intended, not a shortcut being taken with the data.

Full mechanics (Redis key scheme, the two call sites that read/write it, why a Redis outage takes down the cache and the rate limiter and Celery's broker/backend simultaneously) are in the Background Processing and Caching chapter -- not repeated here.

4. What the background job actually does

It is worth being precise about scope here, because the names "Celery worker" and "Celery beat" invite the assumption of a general task queue: **there is exactly one registered Celery task in this codebase**, `run_osint_crawl`, and Celery Beat fires it once every 3600 seconds. It does not process exports, does not re-run failed lookups, and does not do anything else -- those are not implemented, despite being named as in-scope in the Celery module's own docstring.

What that one job actually does, each hour:

1. Looks at `ioc_lookups` for IOCs of a crawler-eligible type (`domain` , `ipv4` , `malware_family` , `threat_actor` , `campaign` , `cve` , `file_name`) that real users looked up in the last 24 hours, capped at 25 IOCs per run.
2. If nothing qualifies, it logs that fact and exits -- normal on a lightly-used instance, not an error.
3. For each qualifying IOC, it re-runs the same "Internet Intelligence Collector" OSINT provider a live investigation would use, and -- only on a successful result -- writes it into the same Redis cache described in §3, so the next analyst to look up that IOC gets a warm cache hit instead of a fresh crawl.

Operationally: if `app-celery_worker-1` or `app-celery_beat-1` is down, no live investigation is affected at all -- the interactive, SSE-streamed lookup pipeline never touches Celery, runs entirely inside the backend process, and has no dependency on either container being up. The only consequence of a stopped worker/beat pair is that the hourly OSINT re-crawl stops happening, which surfaces, if at all, as slightly less warm a cache for recently-investigated IOCs -- nothing breaks or errors. There is no Flower or equivalent monitoring UI bundled; the only way to see this job's history is `docker compose logs app-celery_worker-1 / app-celery_beat-1`, or Celery's own result backend directly (Redis DB `/2`).

5. Backup and restore

The Start Menu's **Backup Database Now** shortcut (`Backup-Database.ps1`) is the supported way to snapshot the platform's data:

- It runs `pg_dump` **inside** the running `app-postgres-1` container via `docker exec`, rather than requiring a separate `psql` / `pg_dump` install on the Windows host -- the container already ships the exact matching server version.
- Output lands in the platform's data directory under a `Backups` subfolder as `postgres-<yyyyMMdd-HH:mm:ss>.sql`.
- The 10 most recent backups are kept; older ones are pruned automatically so the folder doesn't grow unbounded across repeated backups.
- It reads the real `POSTGRES_USER` / `POSTGRES_DB` values out of the install's own `.env` rather than assuming the defaults, so a customized install is still dumped correctly.
- If Postgres isn't running, it says so and exits cleanly (`0`) rather than producing a broken partial file; if `pg_dump` fails or produces an empty file, it deletes the bad output and exits non-zero with the reason logged to the setup log.

The same backup also runs **automatically**, with no separate click, every time the **Configuration** shortcut is used to reconfigure or upgrade an existing install -- the Setup Wizard takes a `pg_dump` snapshot before it re-runs `docker compose up --build` against that

install, so an in-place upgrade is never the first time a backup exists.

Restore is a manual, not scripted, operation. There is no `Restore-Database.ps1` or equivalent Start Menu shortcut in this release -- restoring one of the `.sql` dumps above means, at minimum: stopping `app-backend-1` / `app-celery_worker-1` / `app-celery_beat-1` so nothing is writing to the database mid-restore, then piping the dump back in against `app-postgres-1` (`docker exec -i app-postgres-1 psql -U ioc ioc_intel < postgres-<timestamp>.sql` , adjusting user/db name to match `.env` if customized), then restarting the stopped services. Treat this as an administrator-only, backup-before-you-touch-it operation -- there is no undo once a restore has run.

Neo4j's correlation-graph data and OpenSearch's index are **not** covered by this backup -- only Postgres is. This is a real, disclosed gap rather than an oversight to plan around: the graph and search data can, in the current design, be rebuilt from Postgres-persisted investigation data if lost, but there is no dedicated backup path for either today.

[MISSING FIGURE: backup-database-shortcut-output.png]

6. Monitoring Provider Health and the Executive Dashboard

Two pages, reachable from the same **Dashboard** entry point in the global navigation, are the operational (not per-investigation) view of whether the platform is doing its job:

Provider Health (`GET /api/v1/providers/health` , under the OPERATIONS nav group) is a real, database-backed table -- every registered provider's status, success rate, average latency, and consecutive-failure count, computed from actual recorded call outcomes across four separate rolling windows: 1 hour, 24 hours, 7 days, 30 days. Two guarantees worth checking for explicitly when reviewing this page as an operator, both real bugs found and fixed during this project's own QA pass:

- A provider with **zero real attempts** in a given window shows **Unknown**, never "Healthy" -- silence is never reported as a reassuring signal.
- A provider that correctly reports "nothing found" for an indicator (`NO_DATA` / `UNSUPPORTED_IOC`) counts as a **healthy** outcome, not a failure -- most real-world lookups against most providers legitimately come back empty, since no single provider's dataset covers every indicator. The fix here was measured live: a real provider (OTX) moved from an incorrectly-reported "Degraded"/64% to the correct "Healthy"/100% once this was fixed.

Operationally, this page is the right place to look when an analyst reports "provider X seems broken" -- check whether it's `down` (genuine, repeated failures -- worth checking the credential and the vendor's own status page) versus `unknown` (nobody has actually called it recently, not evidence of a problem) versus a `NO_DATA` -heavy but still- `healthy` provider (working correctly, just has nothing on that particular indicator).

Executive Dashboard (`GET /api/v1/dashboard/kpis` , under the COMMAND nav group) surfaces platform-wide numbers with no hardcoded values: active investigations, critical/high-risk IOCs (30-day window), open cases and open critical cases, average threat score (30-day window), provider health percentage (24h window), and AI success rate (30-day window). The AI success rate deliberately excludes `skipped_no_evidence` outcomes from both numerator and denominator -- a correct decision not to call the AI at all (no provider data to summarize) is not counted as either a success or a failure, and the field reports `null` , not `0` , when there's genuinely no data to compute from. `GET /api/v1/dashboard/executive-summary` layers an AI-written narrative on top of these same numbers, or, if the AI backend is unavailable, a real, number-accurate template narrative -- the response's `"source"` field (`"ai"` or `"template_fallback"`) discloses honestly which one you're looking at, so a degraded AI backend is never silently disguised as a normal AI-generated summary.

A worthwhile daily/weekly habit for an operator: Provider Health for "is intelligence-gathering actually working," the Executive Dashboard for "is the platform doing useful volume, and is the AI backend actually being used successfully" -- and cross-reference both against `GET /api/v1/runtime/audit-log` (who changed what provider/AI configuration, and when) if either one shows an unexplained shift.

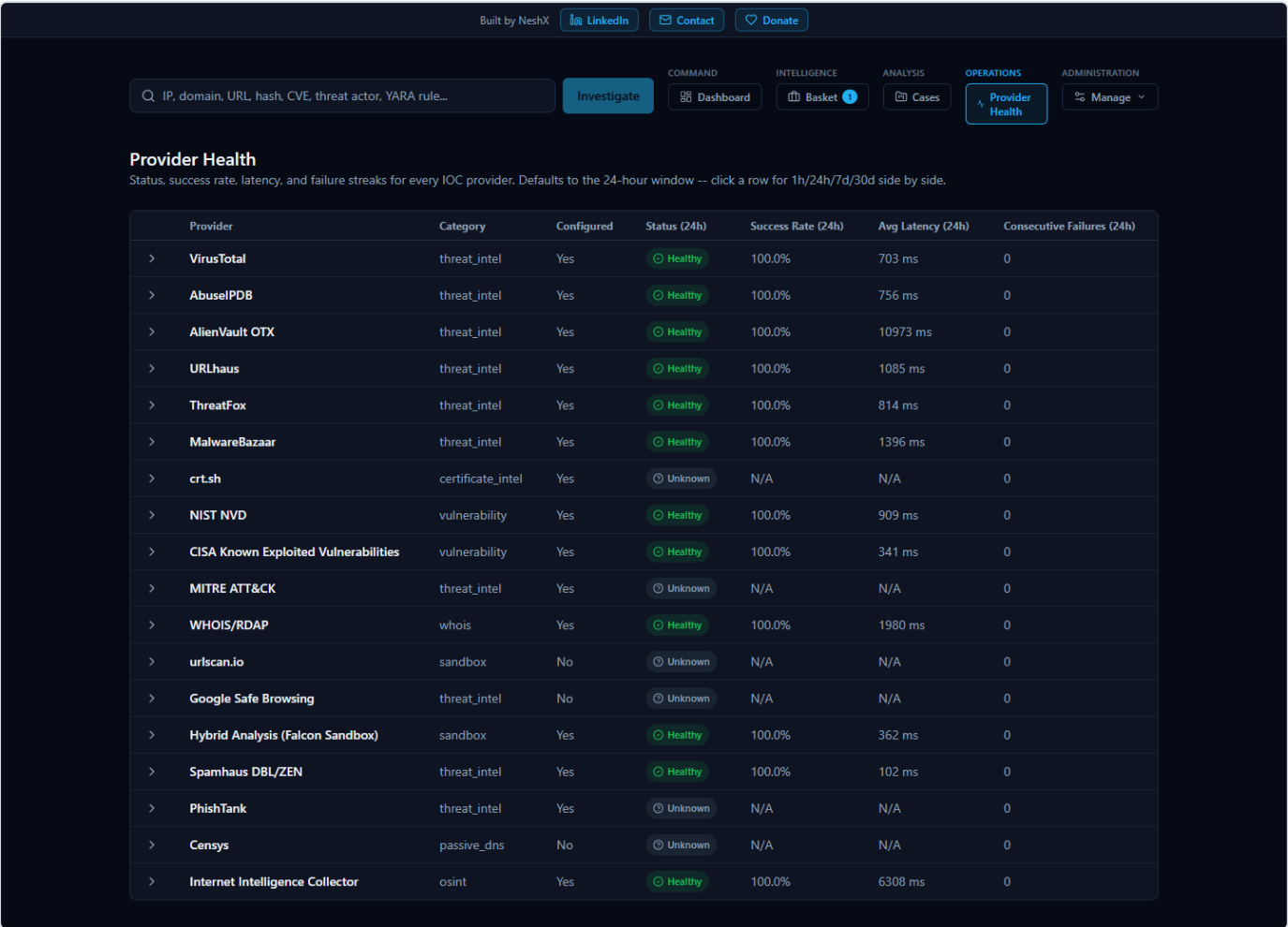


Figure 4 — The Provider Health page's four-window table (1h/24h/7d/30d), reviewed from an operator's perspective for down vs. unknown vs. healthy-but-NO_DATA providers.

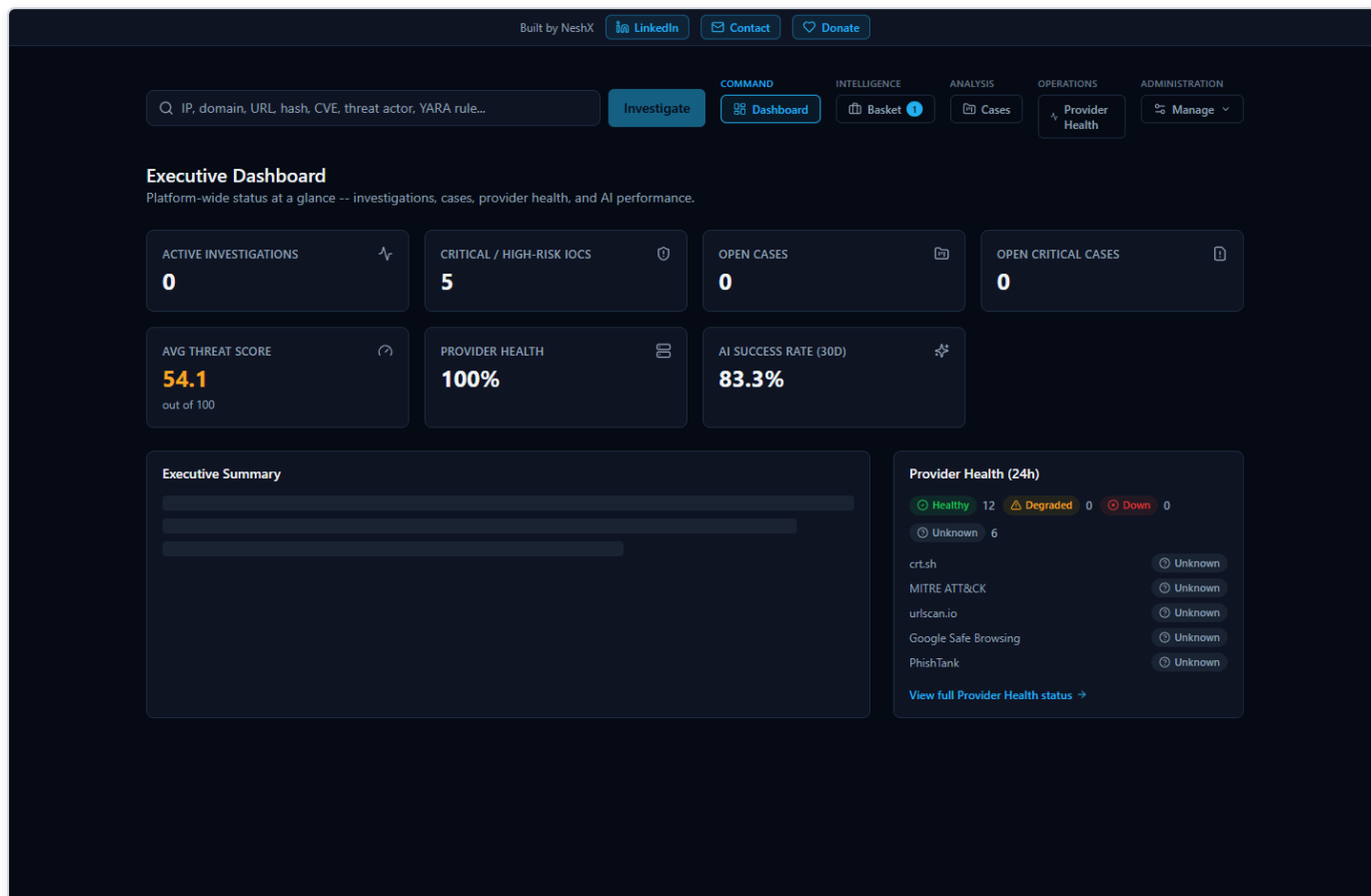


Figure 5 — The Executive Dashboard's KPI tiles and AI-narrative panel, including the "source": "ai" vs. "template_fallback" disclosure.

7. Database connection-pool sizing

This is a real, load-tested fix, not a theoretical tuning note, so it's worth an operator understanding what it protects against.

`app/core/db.py` creates one process-wide async engine per process -- and that module is imported identically by `app-backend-1`, `app-celery_worker-1`, and `app-celery_beat-1`, meaning **three separate engines with three separate connection pools** exist against the same Postgres instance, even though only one of those three processes ever serves concurrent user-facing HTTP traffic.

A real load test against the live containers found a genuine complete-failure mode: 25 concurrent requests to `GET /providers/health` timed out 100% of the time. Two compounding root causes:

1. The endpoint itself issued roughly 306 sequential DB round-trips per request (4 metrics x 4 windows x 18 providers) -- since fixed, consolidated into roughly 19 queries via SQL conditional aggregation.
2. The connection pool was sized to SQLAlchemy's un-tuned library defaults and didn't account for every authenticated request holding **two** database connections simultaneously for its full duration -- one opened by the auth dependency, a second opened separately by the service function it calls. Pool sizing must budget for that multiplier, not one connection per concurrent request.

The fix, still in effect: pool size is now set **per process role**, not identically everywhere. `docker-compose.yml` gives `app-backend-1` an explicit override (`DB_POOL_SIZE=30`, `DB_POOL_MAX_OVERFLOW=20` -- 50 max connections), since it's the only process serving concurrent dashboard/investigation traffic; `app-celery_worker-1` and `app-celery_beat-1` are left on `config.py`'s conservative code-level defaults (`db_pool_size=5`, `db_pool_max_overflow=5` -- 10 max connections each), since between them they run one lightweight hourly task and barely touch the database. Worst case combined total: $50 + 10 + 10 = 70$ connections, comfortably under Postgres's

own default `max_connections=100`. Confirmed live after both fixes: the same 25-concurrent-request test that previously failed 100% of the time completed in 1.6 seconds at 100% success.

An operator does not need to touch this sizing in normal operation -- it's set correctly in `docker-compose.yml` out of the box -- but it's the number to revisit first if `GET /providers/health` or the Executive Dashboard ever starts timing out under real concurrent load again, before assuming it's a code regression.

Summary

Day to day, an operator has four checks available from shallowest to deepest -- `GET /health`, Service Status, Diagnostics, and the application-level Provider Health / Executive Dashboard pages -- and none of them require touching Docker directly. Underneath, the platform is eight `app-*-1` containers, only two of which (`celery_worker`, `celery_beat`) auto-restart; caching is a positive-only, one-hour Redis TTL with no manual invalidation; the only background job is an hourly, 25-IOC-capped OSINT re-crawl that has no bearing on live investigations if it's down; backup is one click and automatic on upgrade, but restore is a manual, documented procedure with no shortcut; and the database connection pool is deliberately sized per process role, with the exact numbers and the real failure they were sized to fix given above.